Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

Nonblocking Concurrent Objects



Outline

- Example nonblocking implementations of concurrent objects
  - Treiber **Stack**
  - Michael & Scott **Queue**
- Important problem settings for nonblocking concurrency
  - The ABA problem
  - Memory reclamation
- Solutions for the above problems
  - Counted pointers
  - Hazard pointers

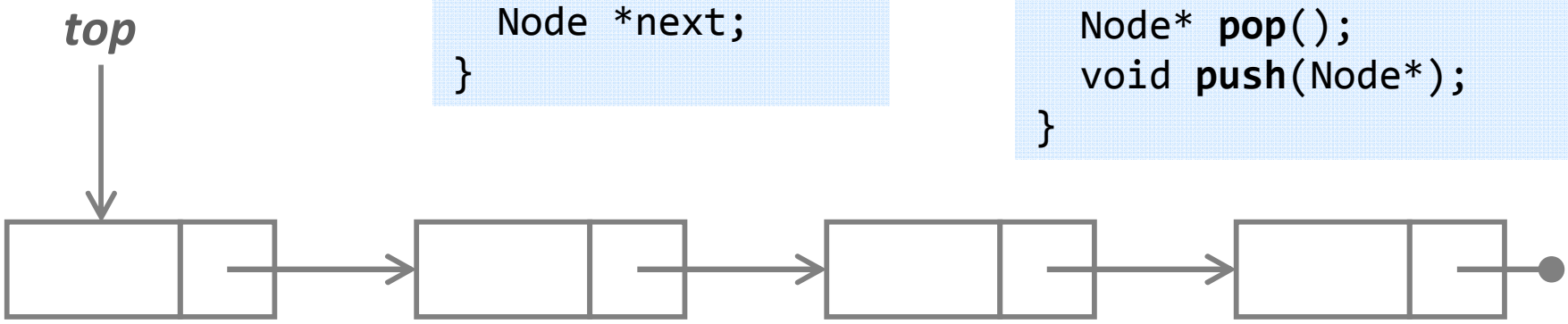
Stacks

- A stack is a basic LIFO (last in first out) data structure
  - Operations are **push**() and **pop**()
  - Can be implemented based on an **array** (**bounded stack**)
  - Or based on a **linked list** (**unbounded stack**)

- A list-based nonblocking stack
  - We assume **push**() and **pop**() operate on nodes (not values)
  - We'll have to atomically update **top** (the top of the stack)

```
class Node {
  T val;
  Node *next;
}
```

```
class Stack {
  atomic Node *top=NULL;
  Node* pop();
  void push(Node*);
}
```



A Nonblocking Concurrent Stack

- Basic idea
  - Use Compare-and-swap to atomically update top of stack (TOS) pointer
  - **push**(): link new node to current top-of-stack, swing TOS to new node
  - **pop**(): follow link from TOS to next node, swing TOS to next node, return previous TOS

```
void push(Node *n) :
  Node *oldtop, *newtop;

  loop
    oldtop = top.load();
    n->next = oldtop;
    newtop = n;
    if(CAS(&top, oldtop, newtop))
      break;
```

```
Node* pop() :
  Node *oldtop, *newtop;

  loop
    oldtop = top.load();
    newtop =
      oldtop?oldtop->next:NULL;
    if(CAS(&top, oldtop, newtop))
      break;

  return oldtop;
```

Discussion

- In this implementation we expect to be passed Node pointers in push() and return them in pop()
  - Thus, **memory management is outside the scope of our implementation**
  - Alternative: pass/return simple values, then we'd have to implement memory management ourselves: **allocating and freeing nodes**
  - In the latter case we'd also have to think about **memory reclamation**: when can we safely re-use memory that may still be in use by other threads (more later)

- Our implementation may appear correct at first inspection, but it has a **serious flaw**:
  - It fails to address the **ABA problem**
  - I.e., it can give wrong results in certain situations

- This algorithm (with some fix for the ABA problem) is called Treiber Stack
  - After Kent Treiber from IBM, who described this algorithm in 1986

The ABA Problem

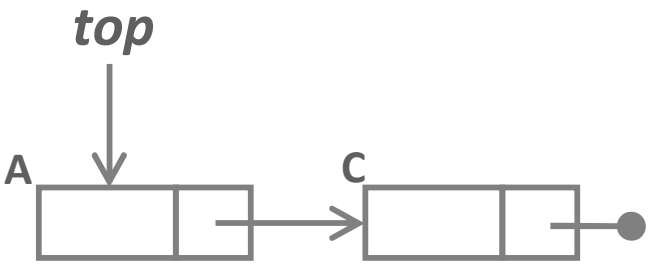
- The ABA problem describes a situation in algorithms that use CAS in a way similar to the following:

```
do {
  oldval = casvar.load(); ❶
  ...
  newval = ...
}
while(!CAS(&casvar, oldval, newval));❷
```

- The problem is that CAS might succeed, even though the value of *casvar* has changed between ❶ and ❷ to some other value and then back again (from “A” to “B” and then back from “B” to “A”).

ABA Problem Example (1)

- Initial Stack



A, **C**: addresses of the first and second node, respectively

```
Node* pop() :
  Node *oldtop, *newtop;

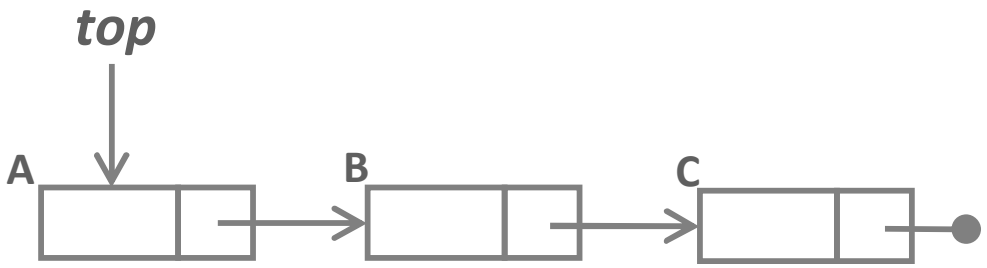
  loop
    oldtop = top.load();
    newtop =
      oldtop?oldtop->next:NULL; ❶
    if(CAS(&top, oldtop, newtop))
      break;

  return oldtop;
```

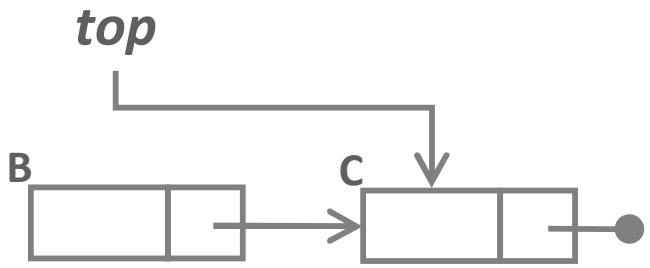
- Thread X is in the process of doing a pop() operation
  - Assume it gets to ❶ and is then delayed (suspended)
  - For thread X oldtop is A, newtop is C
- Assume that while X is suspended, thread Y is completing a pop(), followed by push(B), followed by push(A)

ABA Problem Example (2)

- Thread Y leaves the stack in this state:



- Thread X then resumes and proceeds with the CAS
  - The CAS succeeds, because top is A (again)
  - Thread X swings top to C
  - As a result, node B is lost



Solutions for the ABA Problem

- Counted pointers
  - Make sure all relevant updates to “casvar” have a visible effect
  - struct countedptr = {pointer ptr, count c};
  - Use CAS on the whole structure (including count)
  - Count starts at 0 and all relevant changes increment count
  - Then we’re save from ABA, except for roll-over

- Practical considerations
  - Some machines have 16-Byte CAS, then we can simply use 8 Bytes for pointer, 8 Bytes for “count”
  - If we have only 8 Byte we might still be able to use some bits for “count”
  - E.g., if we are addressing less than 2^64 Bytes, or we are addressing lager chunks than 1 Byte, or if we can use indirect addressing

Fixing the ABA Problem for the Stack (1)

- The only location modified by CAS is the top of stack
  - So we'll use counted pointers there
  - The next pointer can remain a regular pointer

```
class Stack {
  // use counted pointer for top of stack
  atomic <Node*, int> top=<NULL, 0>;
}
```

- For the stack the ABA problem only arises when a combination of push() and pop() operations are executed
  - So it is sufficient to update the count in **either one or both** of these operations

Fixing the ABA Problem for the Stack (2)

```
void push(Node *n) :
  Node *oldtop, *newtop;

  loop
    <oldtop, oldc> = top.load();
    n->next = oldtop;
    newtop = n;
    if(CAS(&top, <oldtop, oldc>,
                 <newtop, oldc>))
      break;
```

```
Node* pop() :
  Node *oldtop, *newtop;

  loop
    <oldtop, oldc> = top.load();
    newtop =
      oldtop?oldtop->next:NULL;
    if(CAS(&top, <oldtop, oldc>,
                 <newtop, oldc+1>))
      break;

  return oldtop;
```

- Every pop() increments the counter

Discussion

- This implementation
  - Solves the ABA problem (absent roll-over)
  - Is lock-free but not wait-free
  - The technique of **elimination** can be used to enable a higher degree of parallelism
- If we change the interface to work with values instead of nodes, we have to deal with the memory reclamation problem too

Stacks and Memory Reclamation (1)

- Passing and returning values changes the interface:

```
class Node {
  T val;
  Node *next;
}
```

```
class Stack {
  atomic Node* top=NULL;

  T pop(); // returns ▲ if empty
  void push(T val);
}
```

```
void push(T val) :
  Node *oldtop, *newtop;
  newtop = new Node(val);

  loop
    oldtop = top.load();
    newtop->next = oldtop;
    if(CAS(&top, oldtop, newtop))
      break;
```

Stacks and Memory Reclamation (2)

```
T pop() :
  Node *oldtop, *newtop;
  T val;

  loop
    oldtop = top.load(); ❶
    newtop =
      oldtop?oldtop->next:NULL; ❸
    if(CAS(&top, oldtop, newtop)
      break;

  val = oldtop?oldtop->val:▲;
  delete(oldtop); ❷
  return val;
```

- Assume thread X executes a pop(), completes line ❶ and is then suspended
- Assume another thread Y executes a pop() to completion. It finds the same top as thread X and finally deletes the node at ❷
- Thread X then resumes and accesses deleted memory when accessing oldtop->next at ❸

Dealing with Memory Reclamation

- We have to find a way to make memory reclamation safe
  - If another thread may still have access to dynamically allocated memory it is not safe to return the memory to the operating system
  - So we need a way to let other threads know which memory is still in use and which one isn't
- Possible solutions
  - Automatic garbage collection (e.g., Java)
  - Atomic reference counting (e.g., std::atomic<std::shared_ptr<T>>, C++20)
  - **Hazard pointers**

Hazard Pointers

- Basic Idea:
  - Each thread makes it known to all other threads which pointers (memory addresses) it considers **hazardous**
  - Before deleting a node, a thread checks all published hazard pointers and only deletes the non-hazard nodes
  - Typically **only one or two pointers are hazardous for a thread at any given time**, so a small shared array of hazard pointers is enough
  - In the stack example one hazard pointer per thread is enough
  - A downside of hazard pointers is that synchronization is needed

Hazard pointers for the Stack (1)

```
T pop() :
  Node *oldtop, *newtop;
  T val;

  loop
    oldtop = top.load();          // mark oldtop hazardous here
    newtop =
      oldtop?oldtop->next:NULL; // hazardous access
    if(CAS(&top, oldtop, newtop)
      break;

  val = oldtop?oldtop->val:▲;    // not a hazardous access
  retire(oldtop);
  return val;
```

- Retire instead of delete
  - Add nodes to a retire list, delete the ones that are not hazardous

Hazard pointers for the Stack (2)

```
// one hazard pointer per thread is
// enough for the stack algorithm
atomic Node* hazardptr[|T|];

T pop() :
  Node *oldtop, *newtop;

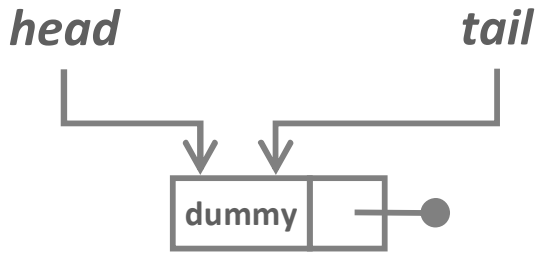
  loop
    oldtop = top.load();
    hazardptr[self] = oldtop;
    if( oldtop!=top.load() )
      continue;
    newtop = oldtop?oldtop->next:NULL; // hazardous access
    if(CAS(&top, oldtop, newtop))
      break;

  // can be omitted if space is not critical
  hazardptr[self]=NULL;
  val = oldtop?oldtop->val:?; // not a hazardous access
  retire(oldtop);

  return val;
```

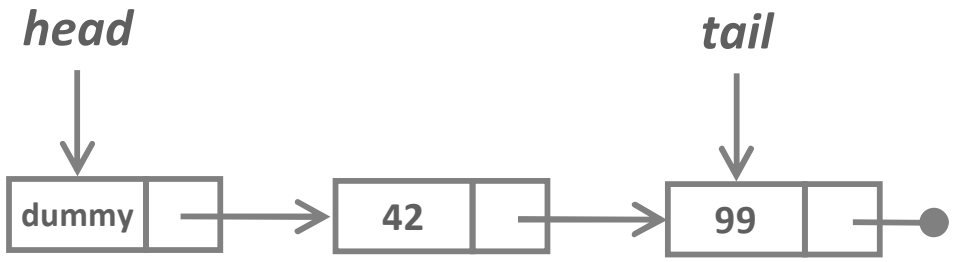
Michael and Scott (M&S) Queue

- A queue is already significantly more complex than a stack
  - Queue has FIFO semantics (first in, first out)
  - Queue has two ends – a **head** and a **tail**
  - **Enqueues at tail**, **dequeues at head** can proceed concurrently
- The M&S queue always keeps a dummy (sentinel) node at the head of the queue
  - The first element to be dequeued is in the node **after** the dummy node
- An empty queue



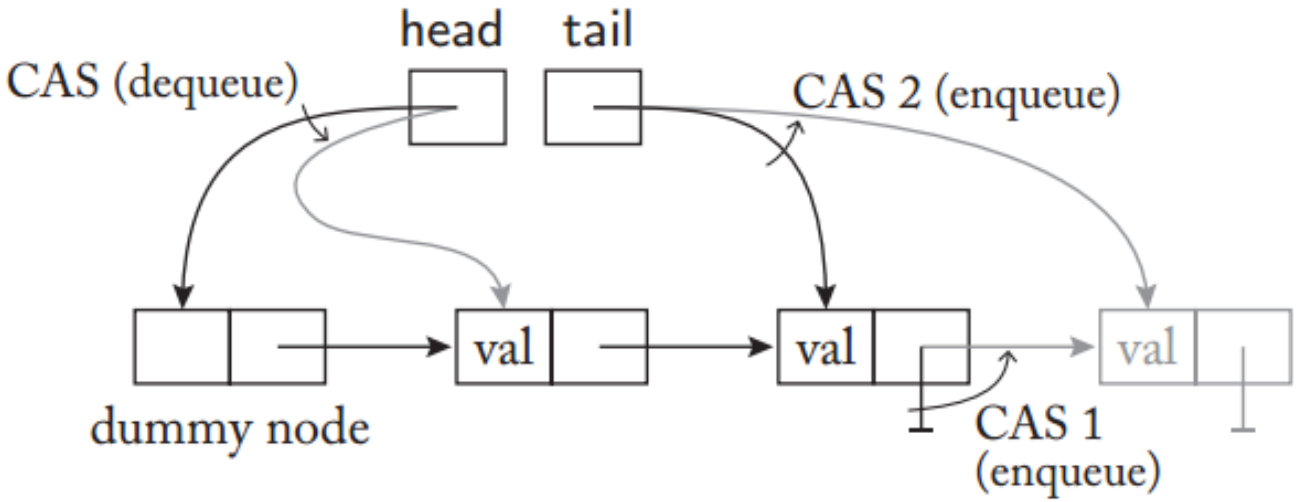
Michael and Scott (M&S) Queue (1)

- A queue with two elements (42 and 99)



- We'll have to update **head**, **tail**, and the **next** pointer of each node atomically
  - For a dequeue(), we'll have to swing the head pointer to the next node using CAS
  - For an enqueue(), we'll have to establish a link from the current tail to the new node using CAS and then we have to swing the tail pointer to the new node using CAS
  - In this version we will work with simple values and deal with memory allocation ourselves

Michael and Scott (M&S) Queue (2)



- Enqueue needs **two** CAS operations that cannot be performed atomically as a whole
  - So any thread must be ready to encounter the queue in an inconsistent state

Michael and Scott (M&S) Queue (3)

```
type ptr = ⟨node* p, int c⟩                    // counted pointer
type node
        value val
        ptr next
class queue
        ptr head
        ptr tail

void queue.init()
        node* n := new node(⊥, null)           // initial dummy node
        head.p := tail.p := n
```

- We use counted pointers again
  - This time for **head**, **tail**, and the **next** pointer in each node
  - Because each of these is updated by CAS and susceptible to the ABA problem

Michael and Scott (M&S) Queue (4)

```
void queue.enqueue(value v):
    node* w := new node(v, null);  fence(W||)        // allocate node for new value
    ptr t, n
    loop
        t := tail.load()                             // counted pointers
        n := t.p→next.load()
        if t = tail.load()                           // are t and n consistent?
            if n.p = null                            // was tail pointing to the last node?
                if CAS(&t.p→next, n, ⟨w, n.c+1⟩)     // try to add w at end of list
                    break                            // success; exit loop
            else                                     // tail was not pointing to the last node
                (void) CAS(&tail, t, ⟨n.p, t.c+1⟩)   // try to swing tail to next node
    (void) CAS(&tail, t, ⟨w, t.c+1⟩)                 // try to swing tail to inserted node
```

- A thread that is not successful in linking in it's node is helping to advance the tail pointer

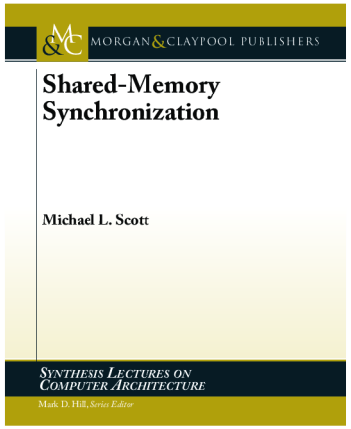
Michael and Scott (M&S) Queue (5)

```
value queue.dequeue():
    ptr h, t, n
    loop
        h := head.load()                                  // counted pointers
        t := tail.load()
        n := h.p→next.load()
        value rtn
        if h = head.load()                                // are h, t, and n consistent?
            if h.p = t.p                                  // is queue empty or tail falling behind?
                if n.p = null return ⊥                    // empty; return failure
                (void) CAS(&tail, t, ⟨n.p, t.c+1⟩)        // tail is falling behind; try to update
            else                                          // no need to deal with tail
                // read value before CAS; otherwise another dequeue might free n
                rtn := n.p→val.load()
                if CAS(&head, h, ⟨n.p, h.c+1⟩)            // try to swing head to next node
                    break                                 // success; exit loop
    fence(‖W)                                             // link node out before deleting!
    free_for_reuse(h.p)                                   // type-preserving
    return rtn                                            // queue was nonempty; return success
```

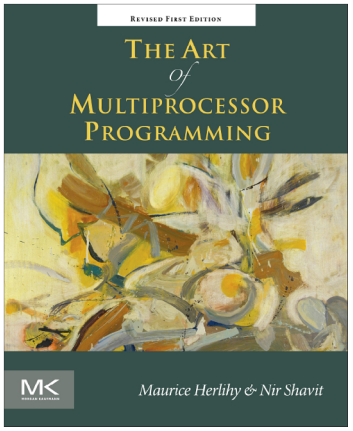
Other Concurrent Objects

- Several other nonblocking concurrent objects have been developed
  - Singly linked lists (Harris & Michael List)
  - Double ended queues
  - Hash tables
  - Skip lists
- See the references at the end of the slides for further pointers

Further Reading



- Michael L. Scott *"Shared-memory synchronization"* Synthesis Lectures on Computer Architecture (2013)
  - Chapter 8 “Nonblocking Algorithms”



- Maurice Herlihy, Nir Shavit *"The Art of Multiprocessor Programming"*
  - Chapter 10 “Concurrent Queues and the ABA Problem”
  - Chapter 11 “Concurrent Stacks and Elimination”